

# Security Audit

## Scout Game (DeFi)

# Table of Contents

|                                   |    |
|-----------------------------------|----|
| Executive Summary                 | 4  |
| Project Context                   | 4  |
| Audit scope                       | 7  |
| Security Rating                   | 8  |
| Intended Smart Contract Functions | 9  |
| Code Quality                      | 10 |
| Audit Resources                   | 10 |
| Dependencies                      | 10 |
| Severity Definitions              | 11 |
| Status Definitions                | 12 |
| Audit Findings                    | 13 |
| Centralisation                    | 16 |
| Conclusion                        | 17 |
| Our Methodology                   | 18 |
| Disclaimers                       | 20 |
| About Hashlock                    | 21 |

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

## Executive Summary

The Scout Game/CharmVerse team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

## Project Context

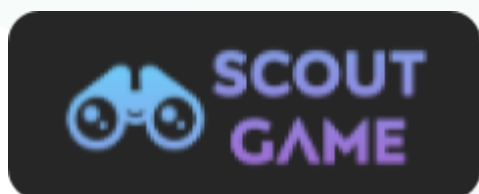
Scout Game is a project created by the CharmVerse team that revolutionizes the Web3 ecosystem by gamifying the support of open-source development. Users act as talent scouts, identifying and backing emerging on-chain builders. Participants collect seasonal Builder NFTs and earn Scout Points as their chosen developers contribute to approved projects through code commits and accepted pull requests. This innovative platform transforms the traditional developer recognition model into an engaging, reward-based experience, fostering a vibrant community that helps propel the on-chain ecosystem forward.

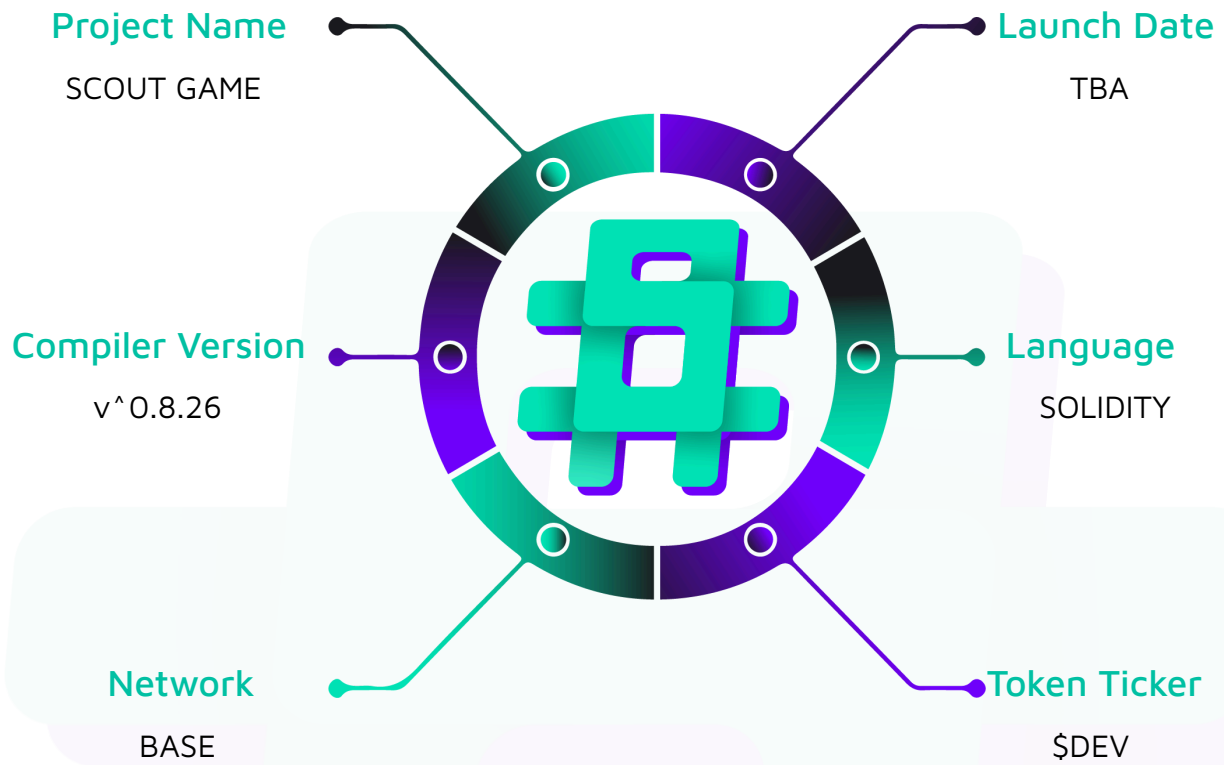
**Project Name:** Scout Game

**Compiler Version:** ^0.8.20

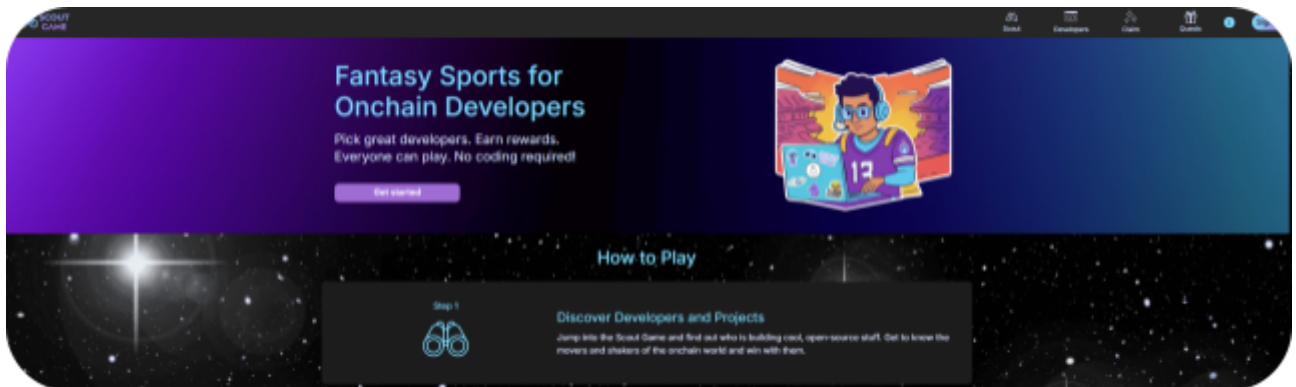
**Website:** [www.scoutgame.xyz](http://www.scoutgame.xyz)

**Logo:**



**Visualised Context:**

## Project Visuals:



## Audit scope

We at Hashlock audited the solidity code within the Scout Game project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing. Hashlock also audited the new GitHub commit hash and no new vulnerabilities were found.

|                                   |                                           |
|-----------------------------------|-------------------------------------------|
| <b>Description</b>                | <b>Scout Game Smart Contracts</b>         |
| <b>Platform</b>                   | <b>Ethereum / Solidity</b>                |
| <b>Audit Date</b>                 | <b>March, 2025</b>                        |
| <b>Contract 1</b>                 | ProtocolEASResolver.sol                   |
| <b>Contract 2</b>                 | ScoutTokenERC20Implementation.sol         |
| <b>Contract 3</b>                 | ScoutTokenERC20Proxy.sol                  |
| <b>Contract 4</b>                 | ScoutProtocolBuilderNFTImplementation.sol |
| <b>Contract 5</b>                 | ScoutProtocolBuilderNFTProxy.sol          |
| <b>Contract 6</b>                 | ScoutProtocolImplementation.sol           |
| <b>Contract 7</b>                 | ScoutProtocolProxy.sol                    |
| <b>Contract 8</b>                 | LockupWeeklyStreamCreator.sol             |
| <b>Contract 9</b>                 | MemoryUtils.sol                           |
| <b>Contract 10</b>                | ScoutProtocolAccessControl.sol            |
| <b>Contract 11</b>                | StringUtils.sol                           |
| <b>Audited GitHub Commit Hash</b> | 52acf98680b8d9b1de7856ec61032e68a0898c49  |
| <b>New Git Hub Commit Hash</b>    | 1e50626a0fc36cc8d19401764e5f462e7380bbc7  |

# Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

## Hashlock found:

1 Medium severity vulnerability

2 Low severity vulnerabilities

1 Gas Optimisation

3 QAs

**Caution:** *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*



## Intended Smart Contract Functions

| Claimed Behaviour                                                                                                                                                  | Actual Behaviour                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <b>ProtocolEASResolver.sol</b><br>An EAS schema resolver that enforces which wallets are allowed to perform attestations against the schemas that use the resolver | Contract achieves this functionality. |
| <b>ScoutTokenERC20Implementation.sol</b><br>Scout Token contract with a cross chain minting/burning functionalities                                                | Contract achieves this functionality. |
| <b>ScoutProtocolBuilderNFTImplementation.sol</b><br>Scout Protocol Builder NFT contract with a purchase functionality in the mint function                         | Contract achieves this functionality. |
| <b>ScoutProtocolImplementation.sol</b><br>Scout Protocol contract with a claiming functionality                                                                    | Contract achieves this functionality. |
| <b>LockupWeeklyStreamCreator.sol</b><br>Vesting contract for \$SCOUT                                                                                               | Contract achieves this functionality. |
| <b>ScoutProtocolStarterNFTImplementation.sol</b><br>Scout Protocol Starter NFT contract with a purchase functionality in the mint function                         | Contract achieves this functionality. |

## Code Quality

This audit scope involves the smart contracts of the Scout Game project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

## Audit Resources

We were given the Scout Game project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

## Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

## Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description                                                                                                                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High         | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| Medium       | Medium-level difficulties should be solved before deployment, but won't result in loss of funds.                                                                                                      |
| Low          | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.                                                                                        |
| Gas          | Gas Optimisations, issues, and inefficiencies                                                                                                                                                         |
| QA           | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.                            |

## Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance        | Description                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resolved</b>     | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue                                                                                    |
| <b>Acknowledged</b> | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| <b>Unresolved</b>   | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.                                                                                                                                                                                               |

# Audit Findings

## Medium

### **[M-01]** ScoutProtocolNFTImplementation#mintTo - Lack of the total supply check

#### **Description**

The contract limits the supply per token id, and the supply must be checked when minting tokens.

However, the `mintTo` function doesn't compare the total supply and to the max supply, and this could cause the total supply to exceed the max supply.

#### **Recommendation**

Add the total supply check in the `mintTo` function, same as the `mint` function.

#### **Status**

Resolved

## Low

### **[L-01]** **ScoutProtocolBuilderNFTImplementation#forwardProceeds** - Return value of `transferFrom` function not checked

#### Description

The `forwardProceeds` function calls the `transferFrom` function of `_paymentToken` contract.

Not all ERC20 implementations perform reverting when there is an error in the `transferFrom`.

If the return value is not checked, a transaction that should be marked as failed may succeed without an actual transfer.

#### Recommendation

Check the return value of the `transferFrom` functions or use the `safeTransferFrom` function instead.

#### Status

Resolved

### **[L-02]** **ScoutProtocolNFTImplementation, ScoutProtocolStarterNFTImplementation** - Important functions lack event emissions

#### Description

Critical functions should emit events to provide transparency and enable off-chain monitoring of important state changes. Without events, it becomes difficult to track and verify contract operations, complicating both user interfaces and protocol monitoring. Instances are shown below.

```
function setBaseUri() // ScoutProtocolNFTImplementation.sol  
function setMinter() // ScoutProtocolNFTImplementation.sol
```

```
function updateBuilderTokenAddress() // ScoutProtocolNFTImplementation.sol
function setProceedsReceiver() // ScoutProtocolNFTImplementation.sol
function updatePriceIncrement() // ScoutProtocolNFTImplementation.sol
function setMaxSupplyPerToken() // ScoutProtocolNFTImplementation.sol
function setBaseUri() // ScoutProtocolStarterNFTImplementation.sol
function setMinter() // ScoutProtocolStarterNFTImplementation.sol
function updateERC20Contract() // ScoutProtocolStarterNFTImplementation.sol
function setUriPrefixAndSuffix() // ScoutProtocolStarterNFTImplementation.sol
function setUriPrefix() // ScoutProtocolStarterNFTImplementation.sol
function setUriSuffix() // ScoutProtocolStarterNFTImplementation.sol
function setProceedsReceiver() // ScoutProtocolStarterNFTImplementation.sol
function updatePriceIncrement() // ScoutProtocolStarterNFTImplementation.sol
function updateBuilderTokenAddress() // ScoutProtocolStarterNFTImplementation.sol
```

## Recommendation

Review your contracts to implement event emissions for critical state variable updates.

## Status

Resolved

## Gas

**[G-01]** **LockupWeeklyStreamCreator** - `SCOUT` and `LOCKUP_TRANCHED` could be made immutable

### Description

The `SCOUT` and `LOCKUP_TRANCHED` variables are only updated in the constructor.

### Recommendation

Make the `SCOUT` and `LOCKUP_TRANCHED` variables immutable.

### Status

Acknowledged



# QA

## [Q-01] Contracts - Floating pragma

### Description

The contracts have `pragma solidity ^0.8.20` and it might allow the contracts to be deployed with a different version than the one used for testing.

Different pragma versions being used in test and mainnet may pose unidentified security issues.

### Recommendation

Specify a specific version of Solidity in the pragma statement.

### Status

Acknowledged

## [Q-02] ScoutProtocolImplementation, ScoutTokenERC20Implementation, ProtocolEASResolver - Unused imports

### Description

The following contracts have unused imports.

`ScoutProtocolImplementation.sol` -> `LockupWeeklyStreamCreator.sol` import

`ScoutTokenERC20Implementation.sol` -> `AccessControlEnumerable.sol` import

`ProtocolEASResolver.sol` -> `SchemaRegistry.sol` import

### Recommendation

Remove unused imports.

### Status

Resolved

**[Q-03] LockupWeeklyStreamCreator** - Comment which represents testnet should be removed

### Description

The `LockupWeeklyStreamCreator` contract has a comment which represents the testnet token address.

```
// sepolia addresses  
IERC20 public SCOUT;  
}
```

Having such comments on production could reduce the code quality.

### Recommendation

Remove the comment.

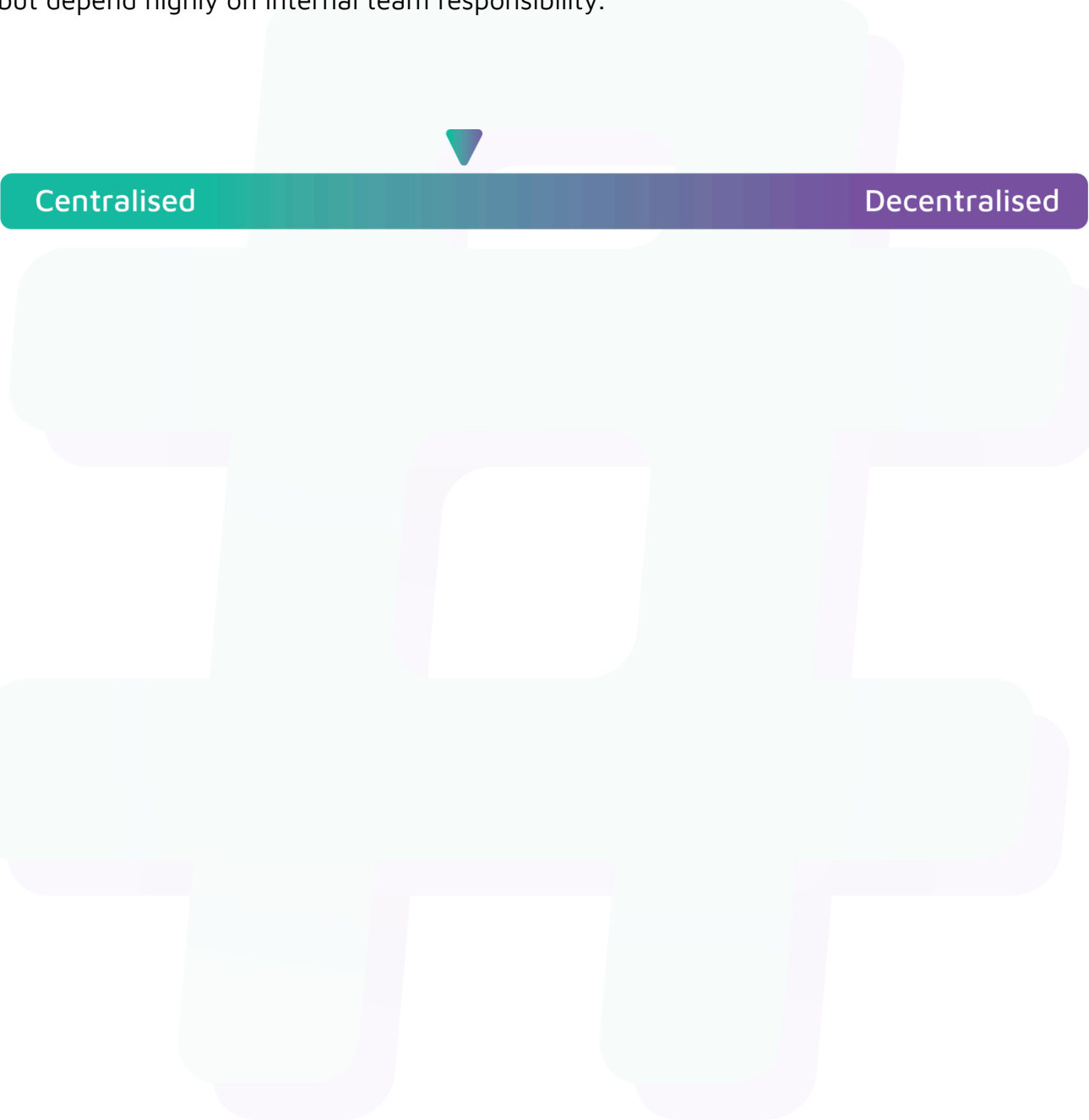
### Status

Resolved

## Centralisation

The Scout Game project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

## Conclusion

After Hashlock's analysis, the Scout Game project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

## Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

### **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

### **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

## About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)





**#hashlock.**